

Integración de Servicios Cloud GCP

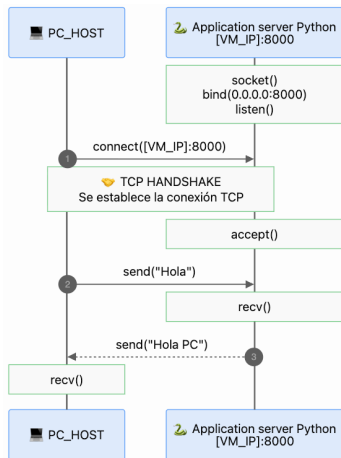
Servidor web y servidor de aplicaciones

Semana 05 - Sesión 10

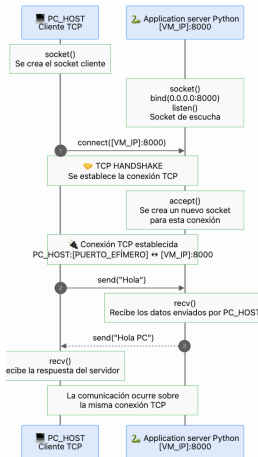
Rel Guzman



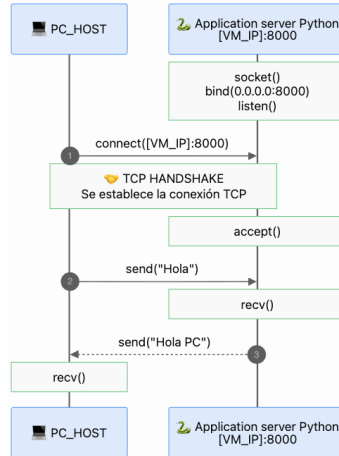
**Universidad
Tecnológica
del Perú**



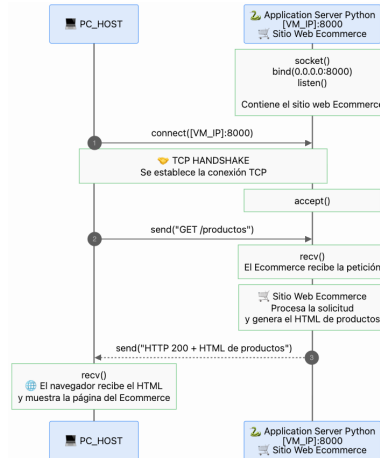
Cliente y servidor hechos con sockets (Sesión 07): el intercambio mínimo sobre TCP.



El mismo intercambio, descrito paso a paso: un socket de escucha, un socket nuevo por conexión.



Visto en resumen: `connect`, `accept` y luego `send/recv` sobre la misma conexión.



La misma mecánica al servicio de un sitio web: recibe GET /productos y devuelve HTML.

Logro de la sesión

Al finalizar, podrás distinguir un **servidor web** de un **servidor de aplicaciones**, ubicar a `nginx` y al programa de Python de la Sesión 09 en ese esquema y explicar por qué en producción se combinan detrás de un proxy inverso.

- ▶ Cuando abres una página, ¿el servidor te envía un archivo ya guardado o arma la respuesta en ese momento?
- ▶ ¿Qué parte del trabajo hizo `nginx` en la práctica de la Sesión 09 y qué parte hizo el programa de Python?
- ▶ ¿Por qué el puerto de la aplicación podía quedar cerrado en el firewall?

¿Para qué te servirá lo aprendido?

Toda aplicación en producción combina un servidor web y un servidor de aplicaciones. Saber qué hace cada uno permite ubicar dónde está una falla y entender por qué se separan.

Es la misma idea de siempre: cada componente con una única responsabilidad clara.

Servidor web y servidor de aplicaciones

El recorrido que montamos

```
Navegador -> nginx :80 -> sitio.py 127.0.0.1:3000  
Navegador <- nginx <----- respuesta generada por Python
```

nginx

Escuchó en el puerto público 80, recibió la solicitud HTTP y la reenvió hacia adentro (proxy inverso).

sitio.py

Ejecutó código Python en 127.0.0.1:3000 y devolvió el HTML que nginx entregó al navegador.

La pregunta de hoy

¿nginx y sitio.py hacen el mismo trabajo?

Idea central

Es un programa que **espera solicitudes HTTP** y devuelve una respuesta. En su forma más básica, esa respuesta es un archivo que ya está guardado en el disco: una página `.html`, una hoja de estilos, una imagen, un archivo de JavaScript. El servidor web solo lo *lee y lo entrega*.

También se encarga de

Aceptar muchas conexiones a la vez, atender HTTPS, registrar los accesos y aplicar límites. Es la **puerta de entrada** pública.

Ejemplos

nginx, Apache HTTP Server. En el curso usamos nginx.

Contenido estático

Se llama **estático** porque el archivo es el mismo para todos los visitantes hasta que alguien lo cambia a mano.

Idea central

Es un programa que **ejecuta el código de una aplicación** para construir la respuesta en el momento de cada solicitud. No lee un archivo terminado: corre un programa que consulta datos, aplica reglas y *genera* el resultado (una página HTML, o datos en formato JSON).

Contenido dinámico

La respuesta puede cambiar según quién pregunta, la hora o lo que haya en la base de datos. Se arma cada vez.

En la Sesión 09

`sitio.py` hizo este papel: ejecutó Python y armó el HTML al vuelo. Usaba `http.server`, que sirve para aprender el flujo, no para producción.

Ejemplos

En Python: Gunicorn o uWSGI ejecutando una aplicación hecha con un framework (Django, Flask, FastAPI).

	Servidor web	Servidor de aplicaciones
Qué devuelve	Un archivo ya guardado	Una respuesta generada en el momento
Qué hace con él	Lo lee y lo entrega	Ejecuta código que lo produce
Tipo de contenido	Estático	Dinámico
Rol habitual	Puerta de entrada, HTTPS, estáticos	Lógica de la aplicación y datos
Ejemplo	nginx, Apache	Gunicorn, uWSGI

Matiz honesto

La frontera no es un muro: un servidor web puede ejecutar algo de lógica y un servidor de aplicaciones puede entregar archivos. Aun así, los roles son distintos y por eso se combinan.

nginx = servidor web

Punto de entrada público en el puerto 80/443.
Entrega archivos estáticos, gestiona HTTPS y,
con `proxy_pass`, reenvía lo demás hacia
adentro.

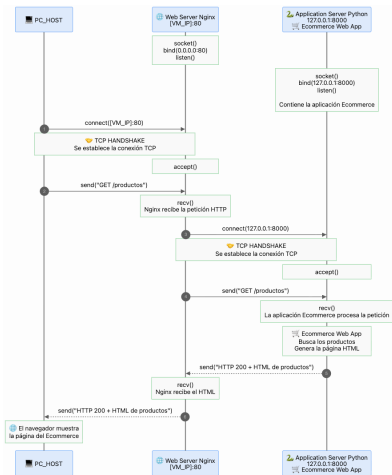
sitio.py = servidor de aplicaciones

Escuchó solo en `127.0.0.1:3000`, ejecutó la
lógica en Python y generó la respuesta
dinámica.

Precisión para producción

Una aplicación real no usa `http.server`. Usa un servidor de aplicaciones especializado (Gunicorn, uWSGI) detrás de `nginx`. El esquema es el mismo; cambia solo la pieza de atrás.

Diagrama de bloques: web + aplicación



nginx (:80) recibe la petición y la reenvía a la aplicación Python interna (:8000), que genera el HTML.

¿Por qué separar las dos piezas?

Cada una en lo suyo

El servidor web está optimizado para muchas conexiones, HTTPS y archivos. El servidor de aplicaciones se concentra en la lógica del negocio.

Seguridad

El puerto de la aplicación queda interno (127.0.0.1). Solo `nginx` se expone. Es lo que comprobaste en la Sesión 09.

Escalar por separado

Puedes poner varias copias de la aplicación detrás del mismo `nginx` y repartir la carga entre ellas.

Mantenimiento

Actualizas o reinicias la aplicación sin tocar la puerta de entrada, y al revés.

- ▶ ¿Qué devuelve un servidor web y qué devuelve un servidor de aplicaciones?
- ▶ ¿Por qué el contenido de un servidor web se llama estático y el de un servidor de aplicaciones, dinámico?
- ▶ En el diagrama de bloques, ¿por qué el servidor de aplicaciones no se expone al exterior?
- ▶ ¿Dónde ubicarías a `nginx` y al `sitio.py` de la Sesión 09 en ese esquema?



gracias



Universidad
Tecnológica
del Perú

- ▶ Documentación de NGINX: *Serving Static Content* y *Reverse Proxy*, docs.nginx.com/nginx/admin-guide.
- ▶ Documentación de Python: módulo `http.server` y nota de "no recomendado para producción", docs.python.org/3/library/http.server.html.
- ▶ Documentación de Gunicorn: *Deploying Gunicorn* (Gunicorn detrás de un proxy como `nginx`), docs.gunicorn.org.
- ▶ Material base: Sesión 09 de este curso; identidad visual UTP y estructura docente reutilizadas sin cambios.